Crabs on camera: project documentation

Gina Brøten

2026-08-11

Contents

1 Background **5**
- 1.1 Project and repository aims 5
- 1.2 About the repository 5
- 1.3 Glossary 6
- 1.4 Before running the scripts 7

2 From raw annotations to YOLO-ready data **9**

Chapter 1

Background

1.1 Project and repository aims

The basis of this project is rooted in the work done in the masters thesis by Brøten [2026]. The aim of the project was to improve monitoring of the data-limited brown crab (*Cancer pagurus*) stock in Norway using machine learning and underwater video surveys.

The aim of this repository is not to push the boundaries of machine learning or showcase new methods, rather to make these methods more approachable for marine scientists working in fisheries applications.

1.2 About the repository

This repository uses the same code, functions, and packages developed for the masters thesis, though some functions have been simplified here to make the pipeline easier to reproduce. The structure of the repository is as follows:

```
crabs-on-camera/
  data/
      annotations/
          test_subset/              # example dataset shipped with the repo
              annotations/
                  instances_default.json   # CVAT/COCO export: labels + crab outlines
              images/default/               # the video frames those annotations belong to
  script/
      raw_to_yolo.ipynb             # converts raw annotations into a YOLO-ready dataset
      build_test_subset.ipynb       # builds a smaller/more diverse test set from a larger one
      annotation_stats.ipynb        # summary statistics on the annotation data
      train.ipynb                   # trains the YOLO model
```

```
    annotation_format              # template CVAT label schema, for annotating your
docs/                              # this documentation
README.md
```

Everything needed to reproduce the pipeline lives in three places: `data/` holds a small data set you can run the scripts against immediately, `script/`holds the actual pipeline as a series of notebooks, and `docs/` is the code documentation.

To try it yourself, clone this repository, either on your own computer for the small example dataset, or on a server with GPU access if you plan to train on a larger dataset.

AI tools (large language models) were used throughout this repository to help generate code, fix syntax, and clean up errors.

1.3 Glossary

- **Object detection**: a computer vision task where a model finds where objects are in an image and identifies what they are — in this project, locating and identifying crabs in video frames.
- **Bounding box**: a rectangular region defined by four coordinates that encloses a specific object within an image or frame.
- **RLE mask** (Run-Length Encoded mask): a pixel-by-pixel outline of an object's exact shape, stored in a compressed format — this is how the original CVAT annotations are drawn, before being converted to bounding boxes.
- **Annotation**: a label marking an object within an image — in CVAT, drawn as a mask or box around each crab.
- **Label**: the concrete tag applied to an annotation — in this context, species (e.g. cod).
- **Class**: the abstract category an object belongs to — in this context, species group/family (e.g. gadoid).
- **IoU** (Intersection over Union): a measure of how much two boxes overlap, used both to remove duplicate boxes during annotation cleanup and later as an evaluation metric.
- **Fold**: one of the K train/validation splits created during K-fold cross-validation.
- **K-fold cross-validation**: splitting the dataset into K equal-sized groups ("folds"). In each round, one fold is held out for validation while the model trains on the rest, repeated K times so every image gets used for both training and validation, just never in the same round. Example: with 5-fold cross-validation, the model is trained and validated 5 separate times, each on a different split, so results don't depend on one lucky (or unlucky) train/validation split.
- **Stratified sampling**: splitting data so each subset keeps roughly the same proportion of a key category as the full dataset. Example: if 30% of

all images contain a crab, stratified sampling makes sure each fold's train and validation sets are also close to 30% crab-present, instead of one fold accidentally ending up almost entirely empty frames.

- **Station leakage**: when images from the same BRUV station appear in both the training and validation sets, letting the model partly be tested on footage from a site it already trained on, this project's K-fold export explicitly prevents it by grouping all images from a station together.
- **Negative example / empty frame**: a frame with no crab in it, deliberately kept in the dataset (rather than discarded) because training on empty frames alongside crab-present ones has been shown to improve performance.
- **Ground truth**: the human-annotated "correct answer" (the CVAT masks/boxes) that a model's predictions are compared against.
- **YAML**: a plain-text file format for configuration settings, structured by indentation rather than punctuation: used here to tell the training script where the data lives and what classes to detect.

1.4 Before running the scripts

1.4.1 Prerequisites

All packages required for data transformation and object detection training are open-source and freely available.

To identify and quantify crabs in the videos, this project uses the object detection framework "You Only Look Once" (YOLO), specifically version 8 [Jocher et al., 2023], which builds on the original YOLO architecture introduced by Redmon et al. [2016]. Several object detection frameworks, including newer YOLO versions, could have been used instead. YOLOv8 was chosen because it is well documented and its common issues are well known, which makes it easier to troubleshoot for beginners in machine learning.

Python must be installed. `raw_to_yolo.ipynb` checks for missing packages and installs them automatically (see Chunk 0).

A server with GPU access is recommended if you plan to train on a large dataset — the small example dataset shipped with this repo can be run locally without one.

If you want to train a detector on your own data, you'll need an annotated dataset structured the same way as the example dataset: an `annotations` folder containing the CVAT/COCO export (`instances_default.json`), and an `images/default` folder containing the matching image files.

1.4.2 Data

The repo only ships a small example/test dataset (`data/annotations/test_subset`) so the scripts can be run and verified without downloading everything.

The dataset used here for testing is from the Institute of Marine Research inshore gill- and fyke-net survey conducted in 2025. The material is from ten Baited Remote Underwater Video (BRUV) stations. For more stats about the annotation composition, run `annotation_stats`.

Annotations are exported from CVAT, as a .JSON file, which is a plain text file storing structured data, in this case annotations, and such as categories (here these are species groups).

The images are labelled with multiple identifiers to make analysis and stratified sampling later simpler.

E.g,. **NO_2025_3007_GarnRuse_Lovund_LO3_RA1_L9_frame_9**, is collected in Norway, in 2025 on the 30.07 on the Garn and Ruse survey (gill- and fyke-net), the location name is Lovund, and is station number is LO3, and the BRUV-rig used is L9, and this is video number 9 from the left camera (L9), and is frame number 9. So if you are applying another type of video-survey or other naming logic please be mindful, though its possible to turn off this stratified sampling in the functions.

If you are annotating your own dataset, a .JSON file for the structure applied in this annotation process can be found in (`script/annotation_format`). For more in depth information about the sampling methods and areas, annotation choices and a discussion of methods can be found in Brøten [2026].

Chapter 2

From raw annotations to YOLO-ready data

This script (`raw_to_yolo.ipnb`) takes raw CVAT annotations and turns them into a dataset ready for training an YOLO object detection model: RLE mask are converted into bounding boxes, and the data is split into five stratified folds for cross-validation.

If you're applying this dataset to other than the shipped example, keep in mind (but not limited to) - File paths: `CONFIG` needs to point at your own annotation JSON and image folder. - Stratified sampling for the k-folds, this assumes your images can be grouped by BRUV station and labeled crab-present/empty; if that doesn't apply to your data, this step may need adjusting or turning off (see Chunk 9).

By the end of the script, each image has a matching `.txt` file listing its bounding box annotations (or an empty file, if there's no crab in it), organized into five train/validation folds ready for object detection training.

1. Setup and loading data (Chunk 0 – 3)

These chunks handle setup: installing any missing Python packages, loading the required libraries, and locating your dataset. They should run without errors, if Chunk 3 can't find your annotation file or image folder, double check the paths.

If you're using your own dataset, update `CONFIG` in Chunk 2 with the correct paths to your annotation JSON and image folder.

2. Converting masks to bounding boxes (Chunk 4)

RLE masks are precise, but training directly on them takes more compute time and is really built for image segmentation, not simple object detection. Since

the goal here is just to find and count crabs (not trace their exact outline), converting to bounding boxes is the better trade-off: they're faster to train on, and sufficient for detection.

RLE masks are also faster to draw in the first place, CVAT's Segment Anything–powered tool makes what's normally a tedious annotation process quicker.

This chunk converts each RLE mask into a tightly-fitted bounding box, and normalizes the box coordinates to the 0–1 range YOLO expects. Two filters are applied at the same time: annotations for any species other than Decapod: crab are discarded, and overlapping duplicate boxes (drawn twice for the same crab) are reduced to one.

3. Linking images to annotations (Chunk 5)

Links each annotation to its actual image file, matching by filename. If you're using your own dataset, make sure image filenames match the identifiers in your annotation JSON exactly, any mismatches get reported as "unmatched," usually caused by a typo, a renamed file, or an image missing from the folder.

Note: frames with no crab in them (negative examples) are deliberately kept in the dataset rather than discarded, training on empty frames as well as crab-present ones has been shown to improve performance [tror sorrenti er riktig kilde]. These get an empty `.txt` label file later, once the data is exported (Chunk 9).

4. Visual quality checks (Chunk 6 – 7)

These chunks are a visual quality check, not a processing step, and nothing here changes the data. Chunk 6 displays the first annotated image with its converted bounding box overlaid: every crab should have a red box tightly wrapping it, with no gaps or extra boxes.

Chunk 7 goes further, placing the original RLE mask and the converted bounding box for the same crab side by side, so you can visually confirm the conversion in Chunk 4 didn't distort or misplace anything before relying on it for training.

5. Location and station helpers (Chunk 8)

Filenames encode which location and BRUV station each image came from, but that information isn't stored as a separate field anywhere, it has to be pulled out of the filename itself. This chunk does that: `extract_location` reads out the site name (e.g. "Lovund"), and `extract_bruv_id` reads out the specific station ID (e.g. "LO3"). It also prints a quick summary: how many images come from each location, and how many of those are crab-present versus empty, worth a glance to check whether your dataset is skewed toward a few sites, since that affects how balanced the folds can be.

If you're using your own dataset with a different filename convention, these two functions will need to be adjusted to match your naming pattern (see the Data section for the convention this project uses).

6. Splitting into stratified, grouped folds (Chunk 9)

This is the core of the export step: it splits the linked dataset into five folds for cross-validation, applying two rules at once.

Grouped: every image from the same BRUV station stays entirely in either training or validation within a fold, never split between the two. If the same station showed up in both, the model could effectively be tested on footage from a site it had already trained on, making it look like it's performing better than it actually would on a genuinely new site.

Stratified: within that constraint, each fold's train/validation split keeps a similar proportion of crab-present vs. empty frames as the full dataset, so no fold accidentally ends up training almost entirely on empty frames.

The function has a `stratify` option: `True` (the default) gives the behavior above; `False` skips the crab/empty balancing and keeps only the station grouping. Turning it off makes sense if your dataset is already fairly balanced, or if the crab/empty ratio varying between folds isn't a concern for you, station grouping still applies either way, so there's no risk of leakage regardless of this setting.

For each fold, this chunk writes the images and label files into `images/train`, `images/val`, `labels/train`, `labels/val`, along with a `dataset.yaml`.

YAML is a plain-text format for storing settings, written as simple, human-readable lines rather than code. YOLO's training tool reads `dataset.yaml` to know where to find the images and labels and what classes to expect. Each fold's file specifies: `path` (that fold's root folder), `train` and `val` (the image subfolders), `nc` (number of classes - 1, since this project only detects crabs), and `names` (mapping each class number to its name, e.g. `0: crab`).

7. Running the export (Chunk 10)

This chunk runs the function from Chunk 9 on your dataset. By default it creates five folds under `data/kfold/`, each structured as:

```
data/kfold/
  fold1/
      images/train/
      images/val/
      labels/train/
      labels/val/
      dataset.yaml
  fold2/ ... fold5/          # same structure as fold1
```

```
master_dataset.yaml
fold_summary.csv
```

along with a `master_dataset.yaml`, a small reference file at the top level listing the class setup (`nc` and `names`, same as each fold's file) and a comment reminding you how to point training at a specific fold (e.g. `yolo train data=fold1/dataset.yaml ...`), and a `fold_summary.csv` with the actual per-fold stats mentioned above.

Once this runs cleanly, no station-overlap warnings, and a crab/empty ratio that looks reasonable in the fold summary, the data is ready for training in `train.ipynb`.

Bibliography

Gina Brøten. Crabs on camera: Improving the monitoring of the brown crab (*Cancer pagurus*) with machine learning and Baited Remote Underwater Video (BRUV). Master's thesis, University of Bergen, 2026. URL https://hdl.handle.net/11250/5537346.

Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics YOLO, January 2023. URL https://github.com/ultralytics/ultralytics.

Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You Only Look Once: Unified, Real-Time Object Detection, May 2016. URL http://arxiv.org/abs/1506.02640. arXiv:1506.02640 [cs.CV].